

**LAPORAN HASIL TUGAS
DATABASE II: ADVANCED DATABASES
CAKRAWALA UNIVERSITY**



Disusun Oleh:

**M. Alwi Perdiansyah
Abdul Muin**

**CAKRAWALA UNIVERSITY
PASAR MINGGU, JAKARTA SELATAN
2025**

PENDAHULUAN

A. Latar Belakang

Coffee shop modern membutuhkan sistem yang andal untuk menjaga ketersediaan bahan, mempercepat transaksi, dan menampilkan laporan penjualan harian. Proyek ini mendemonstrasikan bagaimana PostgreSQL membantu mencapai tujuan tersebut

B. Deskripsi Studi Kasus

Masalah: stok sering tidak sinkron, total pesanan rawan salah input, dan laporan lambat. **Solusi:** trigger stok berbasis resep, perhitungan otomatis total order, view agregat, materialized view menu terlaris, dan transaksi atomic

C. Desain Database & Relasi Tabel utama:

ingredients, inventory, inventory_log, menu_items, recipes, orders, order_items, payments, employees, shifts, audit_logs.

Relasi kunci: recipes(menu_id menu_items, ingredient_id ingredients);
order_items(order_id orders, menu_id menu_items); payments(order_id orders);
inventory(ingredient_id ingredients).

```
DROP SCHEMA IF EXISTS coffee_shop_demo CASCADE;  
CREATE SCHEMA coffee_shop_demo;  
SET search_path TO coffee_shop_demo;
```

```
CREATE TABLE employees (  
  employee_id BIGSERIAL PRIMARY KEY,  
  full_name TEXT NOT NULL,  
  role TEXT NOT NULL CHECK (role IN  
( 'BARISTA', 'CASHIER', 'MANAGER' )),  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

```
CREATE TABLE shifts (  
  shift_id BIGSERIAL PRIMARY KEY,  
  employee_id BIGINT NOT NULL REFERENCES employees(employee_id) ON  
DELETE CASCADE,  
  start_at TIMESTAMPTZ NOT NULL,  
  end_at TIMESTAMPTZ,
```

```

    notes          TEXT
);

CREATE TABLE suppliers (
    supplier_id    BIGSERIAL PRIMARY KEY,
    name           TEXT NOT NULL,
    phone          TEXT,
    email          TEXT,
    address        TEXT
);

CREATE TABLE ingredients (
    ingredient_id   BIGSERIAL PRIMARY KEY,
    name           TEXT NOT NULL UNIQUE,
    unit           TEXT NOT NULL CHECK (unit IN ('g', 'ml', 'pcs')),
    reorder_level  NUMERIC(12,3) NOT NULL DEFAULT 0
);

CREATE TABLE inventory (
    inventory_id    BIGSERIAL PRIMARY KEY,
    ingredient_id    BIGINT NOT NULL UNIQUE REFERENCES
ingredients(ingredient_id) ON DELETE CASCADE,
    on_hand        NUMERIC(12,3) NOT NULL DEFAULT 0,
    on_order       NUMERIC(12,3) NOT NULL DEFAULT 0,
    last_updated   TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE inventory_log (
    log_id          BIGSERIAL PRIMARY KEY,
    ingredient_id    BIGINT NOT NULL REFERENCES ingredients(ingredient_id) ON
DELETE CASCADE,
    direction       SMALLINT NOT NULL CHECK (direction IN (1,-1)), -- 1=in,
-1=out
    qty            NUMERIC(12,3) NOT NULL CHECK (qty > 0),
    reason          TEXT NOT NULL, -- SALE/ADJUSTMENT/PURCHASE/REFUND
    ref_id          BIGINT,
    logged_at       TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE menu_items (
    menu_id         BIGSERIAL PRIMARY KEY,
    name           TEXT NOT NULL,
    category        TEXT NOT NULL CHECK (category IN ('COFFEE', 'NON-
COFFEE', 'FOOD', 'OTHER')),
    base_price      NUMERIC(12,2) NOT NULL CHECK (base_price >= 0),
    active          BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE recipes (
    menu_id         BIGINT NOT NULL REFERENCES menu_items(menu_id) ON DELETE
CASCADE,
    ingredient_id    BIGINT NOT NULL REFERENCES ingredients(ingredient_id) ON
DELETE CASCADE,
    qty_per_unit    NUMERIC(12,3) NOT NULL CHECK (qty_per_unit > 0),
    PRIMARY KEY (menu_id, ingredient_id)
);

```

```

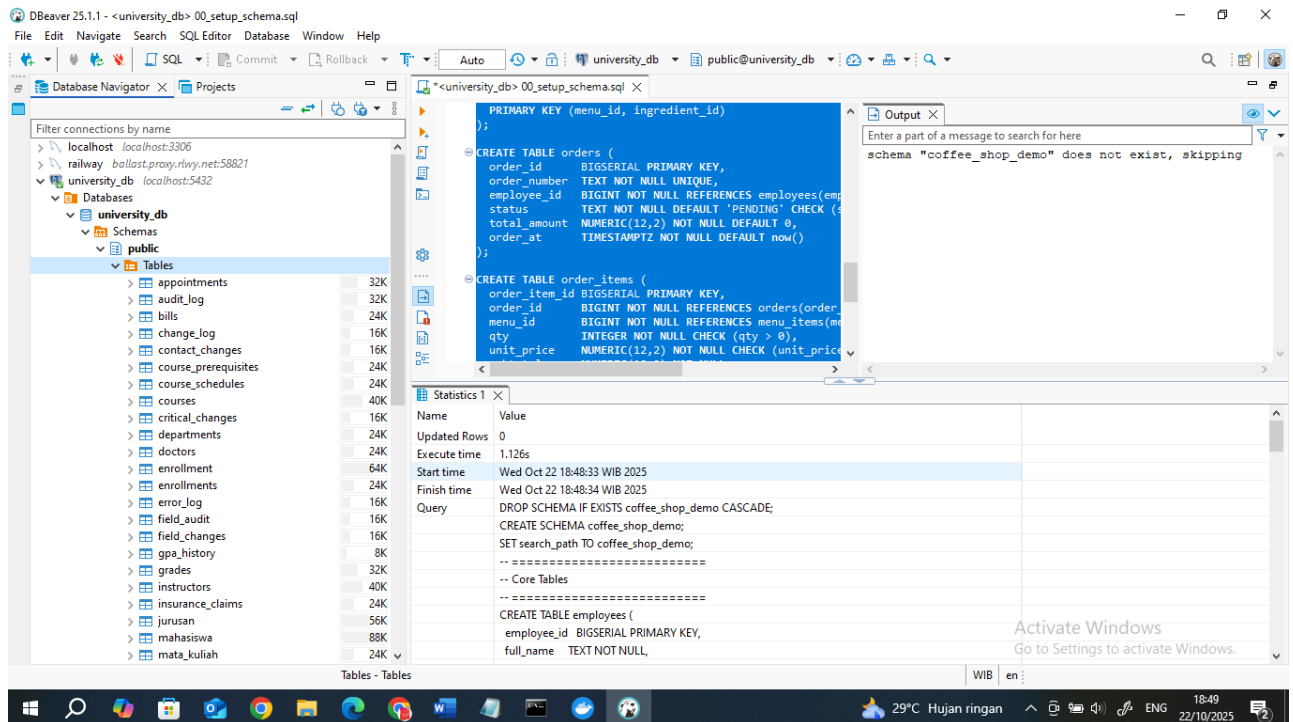
CREATE TABLE orders (
  order_id      BIGSERIAL PRIMARY KEY,
  order_number  TEXT NOT NULL UNIQUE,
  employee_id   BIGINT NOT NULL REFERENCES employees(employee_id),
  status        TEXT NOT NULL DEFAULT 'PENDING' CHECK (status IN
('PENDING','PAID','CANCELLED')),
  total_amount  NUMERIC(12,2) NOT NULL DEFAULT 0,
  order_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE order_items (
  order_item_id BIGSERIAL PRIMARY KEY,
  order_id       BIGINT NOT NULL REFERENCES orders(order_id) ON DELETE
CASCADE,
  menu_id        BIGINT NOT NULL REFERENCES menu_items(menu_id),
  qty            INTEGER NOT NULL CHECK (qty > 0),
  unit_price     NUMERIC(12,2) NOT NULL CHECK (unit_price >= 0),
  subtotal       NUMERIC(12,2) NOT NULL
);

CREATE TABLE payments (
  payment_id     BIGSERIAL PRIMARY KEY,
  order_id       BIGINT NOT NULL REFERENCES orders(order_id) ON DELETE
CASCADE,
  amount         NUMERIC(12,2) NOT NULL CHECK (amount >= 0),
  method         TEXT NOT NULL CHECK (method IN
('CASH','CARD','EWALLET','TRANSFER')),
  paid_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE audit_logs (
  log_id         BIGSERIAL PRIMARY KEY,
  table_name     TEXT NOT NULL,
  action         TEXT NOT NULL,
  row_id         BIGINT NOT NULL,
  log_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  details        JSONB
);

```

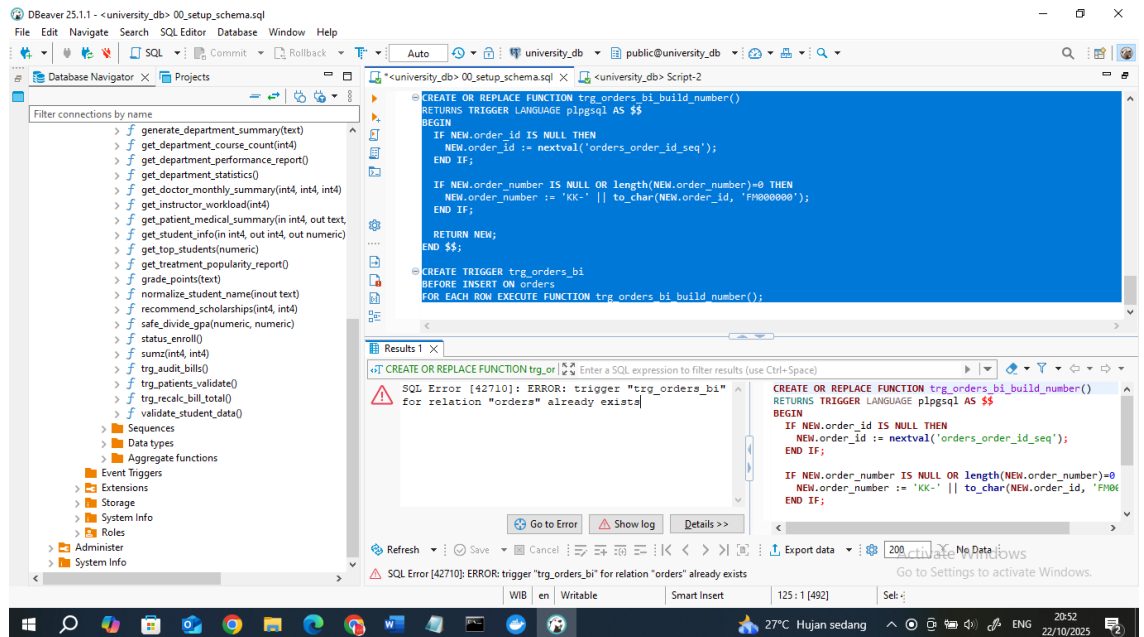


D. Implementasi Triggers

1. orders_bi_build_number

Trigger ini secara otomatis membuat nomor pesanan (order_number) dengan format seperti “KK-000001” setiap kali data baru dimasukkan ke tabel orders.

Tujuannya agar setiap pesanan memiliki kode unik dan berurutan tanpa perlu input manual dari kasir atau sistem lain.



Disini ada keterangan :

SQL Error [42710]: ERROR: trigger "trg_orders_bi" for relation "orders" already exists

Ini dikarenakan mencoba membuat trigger dengan nama yang sama (trg_orders_bi) pada tabel orders, padahal trigger itu sudah ada sebelumnya di database.

PostgreSQL gak ngizinin dua trigger dengan nama sama pada satu tabel.

Maka solusinya Hapus dulu trigger lama sebelum buat ulang

DROP TRIGGER IF EXISTS trg_orders_bi ON orders;

Setelah itu dicoba kembali

```
CREATE OR REPLACE FUNCTION trg_orders_bi_build_number()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.order_id IS NULL THEN
        NEW.order_id := nextval('orders_order_id_seq');
    END IF;

    IF NEW.order_number IS NULL OR length(NEW.order_number)=0 THEN
        NEW.order_number := 'KK-' || to_char(NEW.order_id, 'FM000000');
    END IF;

    RETURN NEW;
END $$;
```

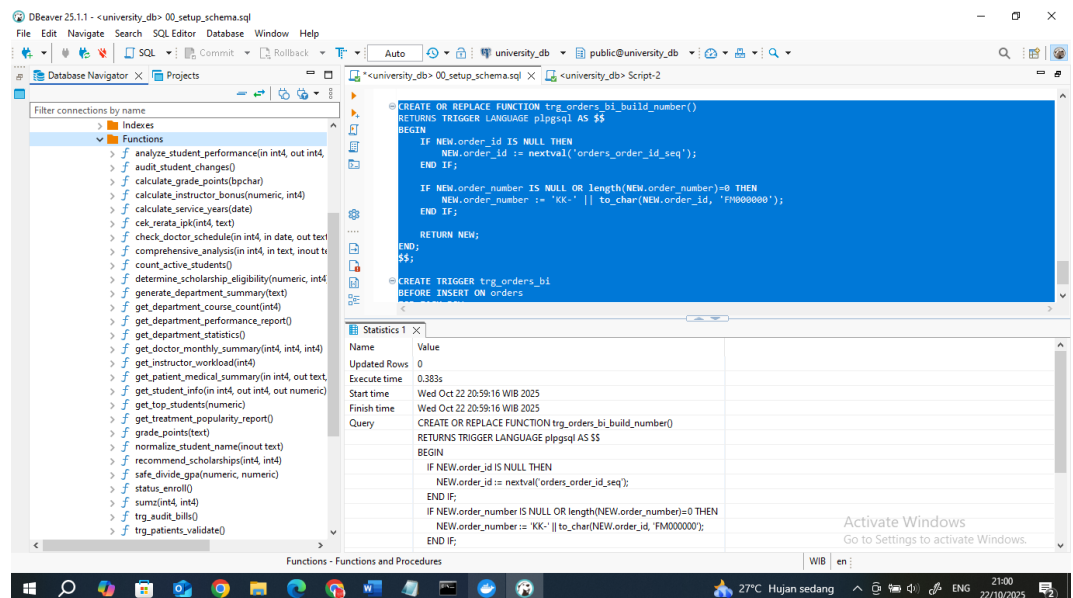
```

NEW.order_number := 'KK-' || to_char(NEW.order_id, 'FM000000');
END IF;

RETURN NEW;
END;
$$;

CREATE TRIGGER trg_orders_bi
BEFORE INSERT ON orders
FOR EACH ROW
EXECUTE FUNCTION trg_orders_bi_build_number();

```



Contoh penggunaan:

Masukkan data ke tabel order

```

INSERT INTO orders (employee_id, customer_name, total_amount)
VALUES (1, 'Ucok Baba', 50000),
(1, 'Siti Rahma', 75000);

```

Cek hasilnya :

```

SELECT order_id, order_number, customer_name, total_amount, employee_id
FROM orders
ORDER BY order_id;

```

order_id	order_number	customer_name	total_amount	employee_id
4	KK-000004	Ucok Baba	50,000	1
5	KK-000005	Siti Rahma	75,000	1

- Saat INSERT dijalankan, PostgreSQL otomatis memanggil **trigger trg_orders_bi**.
- Fungsi `trg_orders_bi_build_number()` menghasilkan **kode unik KK-000004** berdasarkan `order_id`.
- Jadi gak perlu lagi nulis `order_number` manual setiap kali transaksi baru dibuat.

2. `order_items_biu_calc` dan `order_items_bd_calc`

Kedua trigger ini berfungsi untuk menghitung subtotal tiap item dan total keseluruhan pesanan secara otomatis:

Saat INSERT atau UPDATE, sistem langsung menghitung subtotal ($\text{qty} \times \text{unit_price}$) dan memperbarui `total_amount` pada tabel `orders`.

Saat DELETE, sistem mengurangi nilai `total_amount` sesuai item yang dihapus.

Dengan begitu, nilai total pesanan selalu akurat dan konsisten, tanpa harus dihitung manual.

```
CREATE OR REPLACE FUNCTION trg_order_items_biu_calc()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    target_order BIGINT;
BEGIN
    IF TG_OP IN ('INSERT', 'UPDATE') THEN
        NEW.subtotal := COALESCE(NEW.qty, 0) * COALESCE(NEW.unit_price, 0);
        target_order := NEW.order_id;
    ELSE
        target_order := OLD.order_id;
    END IF;
END;
```



```

END IF;

UPDATE orders o
SET total_amount = COALESCE((SELECT SUM(oi.subtotal) FROM order_items oi WHERE
oi.order_id = target_order),0)
WHERE o.order_id = target_order;

RETURN COALESCE(NEW, OLD);
END $$;

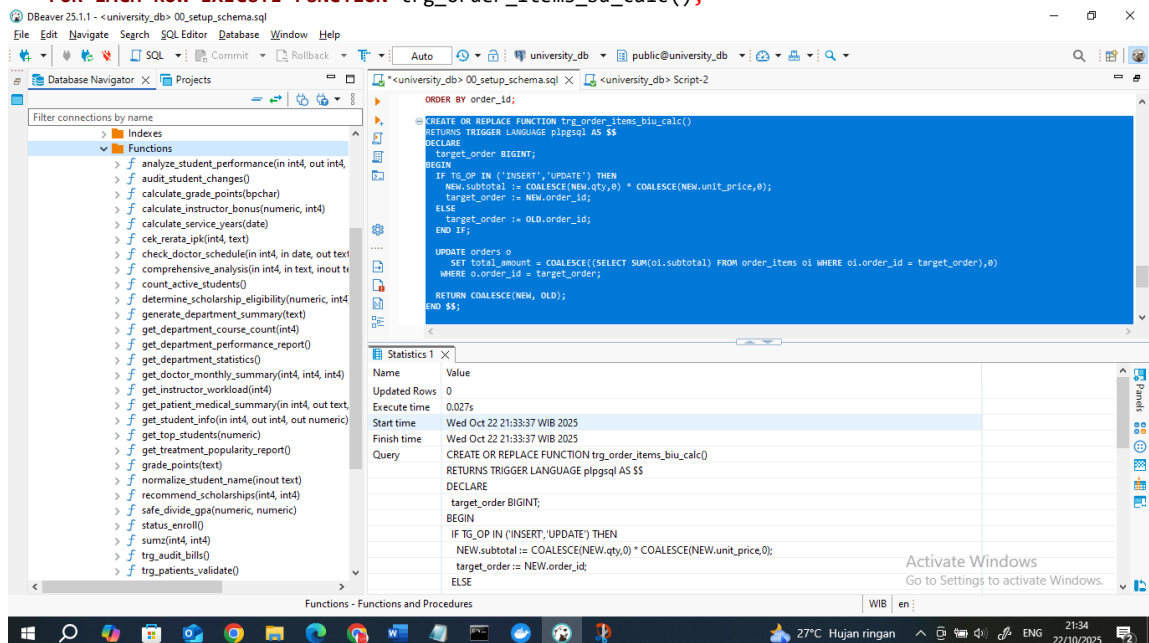
CREATE TRIGGER trg_order_items_bi
BEFORE INSERT ON order_items
FOR EACH ROW EXECUTE FUNCTION trg_order_items_biu_calc();

CREATE TRIGGER trg_order_items_bu
BEFORE UPDATE ON order_items
FOR EACH ROW EXECUTE FUNCTION trg_order_items_biu_calc();

CREATE OR REPLACE FUNCTION trg_order_items_bd_calc()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
UPDATE orders o
SET total_amount = COALESCE((SELECT SUM(oi.subtotal) FROM order_items oi WHERE
oi.order_id = OLD.order_id),0)
WHERE o.order_id = OLD.order_id;
RETURN OLD;
END $$;

CREATE TRIGGER trg_order_items_bd
AFTER DELETE ON order_items
FOR EACH ROW EXECUTE FUNCTION trg_order_items_bd_calc();

```



Contoh Penggunaan:

```

-- Buat order baru dan ambil id-nya
WITH new_order AS (
INSERT INTO orders (employee_id, customer_name) -- kolom customer_name opsional;
tambahkan kalau ada
VALUES (
(SELECT employee_id FROM employees WHERE full_name = 'Admin Kasir' LIMIT 1),
'Ucok Baba'
)
RETURNING order_id
)

```

```
-- Tambahkan 2 item untuk order tersebut
INSERT INTO order_items (order_id, menu_id, qty, unit_price)
SELECT
    order_id,
    (SELECT menu_id FROM menu_items WHERE name='Latte' LIMIT 1),
    2,
    (SELECT base_price FROM menu_items WHERE name='Latte' LIMIT 1)
FROM new_order
UNION ALL
SELECT
    order_id,
    (SELECT menu_id FROM menu_items WHERE name='Espresso' LIMIT 1),
    1,
    (SELECT base_price FROM menu_items WHERE name='Espresso' LIMIT 1)
FROM new_order;
```

Lalu check hasilnya

```
SELECT * FROM orders ORDER BY order_id DESC LIMIT 3;
SELECT oi.order_item_id, oi.order_id, mi.name AS menu_name, oi.qty, oi.unit_price,
oi.subtotal
FROM order_items oi
JOIN menu_items mi ON mi.menu_id = oi.menu_id
ORDER BY oi.order_item_id DESC LIMIT 5;
```

The screenshot shows a database application interface with two tables displayed in a grid view.

Table 1: orders

order_id	order_number	employee_id	status	total_amount	order_at	customer_name
6	KK-000006	1	PENDING	50,000	2025-10-22 21:55:28.256 +0700	Ucok Baba
5	KK-000005	1	PENDING	75,000	2025-10-22 21:23:43.207 +0700	Siti Rahma
4	KK-000004	1	PENDING	50,000	2025-10-22 21:23:43.207 +0700	Ucok Baba

Table 2: order_items

order_item_id	order_id	menu_name	qty	unit_price	subtotal
6	6	Espresso	1	15,000	15,000
5	6	Latte	2	25,000	50,000

Fungsi `trg_order_items_biu_calc()` bekerja → subtotal langsung muncul.

Fungsi `trg_order_items_bd_calc()` aktif → kalau ada item dihapus, total langsung menyesuaikan.

Relasi `orders` → `order_items` berjalan baik → Foreign Key valid.

3. inventory_apply_recipe

Trigger ini berjalan setelah proses INSERT, UPDATE, atau DELETE pada tabel order_items.

Fungsinya adalah menyesuaikan stok bahan baku berdasarkan resep dari menu yang dijual:

Saat ada pesanan baru, stok bahan akan otomatis berkurang sesuai jumlah item yang dipesan.

Jika pesanan dibatalkan atau diubah, stok bahan akan dikembalikan (revert) sebagian atau seluruhnya.

Trigger ini juga memeriksa apakah stok mencukupi. Kalau stok tidak cukup, sistem akan menampilkan pesan “Insufficient stock” dan transaksi otomatis dibatalkan (ROLLBACK).

```
CREATE OR REPLACE FUNCTION trg_inventory_apply_recipe()  
RETURNS TRIGGER LANGUAGE plpgsql AS $$  
DECLARE  
    rec RECORD;  
BEGIN  
    IF TG_OP = 'INSERT' THEN  
        FOR rec IN  
            SELECT r.ingredient_id, (r.qty_per_unit * NEW.qty) AS delta  
            FROM recipes r WHERE r.menu_id = NEW.menu_id  
        LOOP  
            UPDATE inventory  
            SET on_hand = ROUND(on_hand - rec.delta, 3), last_updated = now()  
            WHERE ingredient_id = rec.ingredient_id;  
            INSERT INTO inventory_log(ingredient_id, direction, qty, reason, ref_id)  
            VALUES (rec.ingredient_id, -1, ROUND(rec.delta,3), 'SALE', NEW.order_id);  
  
            IF (SELECT on_hand FROM inventory WHERE ingredient_id=rec.ingredient_id) < 0 THEN  
                RAISE EXCEPTION 'Insufficient stock for ingredient_id=%', rec.ingredient_id;  
            END IF;  
        END LOOP;  
        RETURN NEW;  
  
    ELSIF TG_OP = 'UPDATE' THEN  
        IF NEW.menu_id = OLD.menu_id THEN  
            FOR rec IN  
                SELECT r.ingredient_id, (r.qty_per_unit * (NEW.qty - OLD.qty)) AS delta  
                FROM recipes r WHERE r.menu_id = NEW.menu_id  
            LOOP  
                UPDATE inventory  
                SET on_hand = ROUND(on_hand - rec.delta, 3), last_updated = now()  
                WHERE ingredient_id = rec.ingredient_id;  
                INSERT INTO inventory_log(ingredient_id, direction, qty, reason, ref_id)  
                VALUES (rec.ingredient_id, CASE WHEN rec.delta>0 THEN -1 ELSE 1 END,  
                ROUND(ABS(rec.delta),3), 'SALE_ADJUST', NEW.order_id);
```

```

        IF (SELECT on_hand FROM inventory WHERE ingredient_id=rec.ingredient_id) < 0 THEN
            RAISE EXCEPTION 'Insufficient stock for ingredient_id=% (adjust)',
rec.ingredient_id;
        END IF;
    END LOOP;
ELSE
    -- Revert OLD
    FOR rec IN
        SELECT r.ingredient_id, (r.qty_per_unit * OLD.qty) AS delta
        FROM recipes r WHERE r.menu_id = OLD.menu_id
    LOOP
        UPDATE inventory
            SET on_hand = ROUND(on_hand + rec.delta, 3), last_updated = now()
            WHERE ingredient_id = rec.ingredient_id;
        INSERT INTO inventory_log(ingredient_id, direction, qty, reason, ref_id)
            VALUES (rec.ingredient_id, 1, ROUND(rec.delta,3), 'SALE_REVERT', NEW.order_id);
    END LOOP;

    -- Apply NEW
    FOR rec IN
        SELECT r.ingredient_id, (r.qty_per_unit * NEW.qty) AS delta
        FROM recipes r WHERE r.menu_id = NEW.menu_id
    LOOP
        UPDATE inventory
            SET on_hand = ROUND(on_hand - rec.delta, 3), last_updated = now()
            WHERE ingredient_id = rec.ingredient_id;
        INSERT INTO inventory_log(ingredient_id, direction, qty, reason, ref_id)
            VALUES (rec.ingredient_id, -1, ROUND(rec.delta,3), 'SALE', NEW.order_id);

        IF (SELECT on_hand FROM inventory WHERE ingredient_id=rec.ingredient_id) < 0 THEN
            RAISE EXCEPTION 'Insufficient stock for ingredient_id=% (switch)',
rec.ingredient_id;
        END IF;
    END LOOP;
    END IF;
    RETURN NEW;

ELSIF TG_OP = 'DELETE' THEN
    FOR rec IN
        SELECT r.ingredient_id, (r.qty_per_unit * OLD.qty) AS delta
        FROM recipes r WHERE r.menu_id = OLD.menu_id
    LOOP
        UPDATE inventory
            SET on_hand = ROUND(on_hand + rec.delta, 3), last_updated = now()
            WHERE ingredient_id = rec.ingredient_id;
        INSERT INTO inventory_log(ingredient_id, direction, qty, reason, ref_id)
            VALUES (rec.ingredient_id, 1, ROUND(rec.delta,3), 'REFUND', OLD.order_id);
    END LOOP;
    RETURN OLD;
END IF;

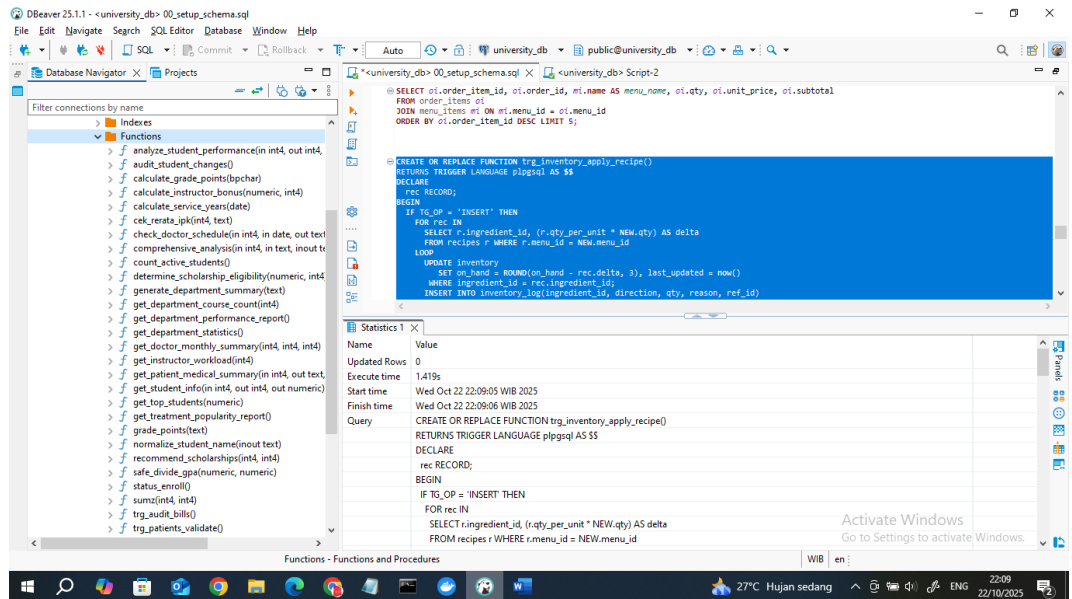
RETURN COALESCE(NEW, OLD);
END $$;

CREATE TRIGGER trg_inventory_ai
AFTER INSERT ON order_items
FOR EACH ROW EXECUTE FUNCTION trg_inventory_apply_recipe();

CREATE TRIGGER trg_inventory_au
AFTER UPDATE ON order_items
FOR EACH ROW EXECUTE FUNCTION trg_inventory_apply_recipe();

CREATE TRIGGER trg_inventory_ad
AFTER DELETE ON order_items
FOR EACH ROW EXECUTE FUNCTION trg_inventory_apply_recipe();

```



Cara penggunaannya

1. Siapkan bahan & stok awal

```
-- Tambahkan bahan-bahan
;
INSERT INTO ingredients (name, unit, reorder_level)
VALUES
('Kopi Bubuk', 'g', 100),
('Susu Cair', 'ml', 500),
('Gula', 'g', 200);

-- Tambahkan stok awal ke inventory
TRUNCATE TABLE inventory, ingredients RESTART IDENTITY CASCADE;

INSERT INTO ingredients (name, unit, reorder_level)
VALUES
('Kopi Bubuk', 'g', 100),
('Susu Cair', 'ml', 500),
('Gula', 'g', 200);
```

2. Buat menu dan resepnya

```
-- Tambahkan menu
INSERT INTO menu_items (name, category, base_price)
VALUES
('Latte', 'COFFEE', 25000),
('Espresso', 'COFFEE', 15000);

-- Definisikan resep menu
-- 1 Latte = 10g Kopi Bubuk + 200ml Susu + 10g Gula
INSERT INTO recipes (menu_id, ingredient_id, qty_per_unit)
VALUES
(1, 1, 10), -- Kopi Bubuk
(1, 2, 200), -- Susu
(1, 3, 10); -- Gula

-- 1 Espresso = 10g Kopi Bubuk + 5g Gula
INSERT INTO recipes (menu_id, ingredient_id, qty_per_unit)
VALUES
(2, 1, 10),
```

```
(2, 3, 5);
```

3. Buat order dan tambahkan item (trigger aktif di sini)

```
-- Buat order  
INSERT INTO orders (employee_id, customer_name)  
VALUES (1, 'Budi');
```

Sekarang tambahkan item pesanan:

```
-- Tambahkan 1 Latte dan 1 Espresso ke order  
INSERT INTO order_items (order_id, menu_id, qty, unit_price)  
VALUES  
(4, 1, 1, 25000), -- Latte  
(5, 2, 1, 15000); -- Espresso
```

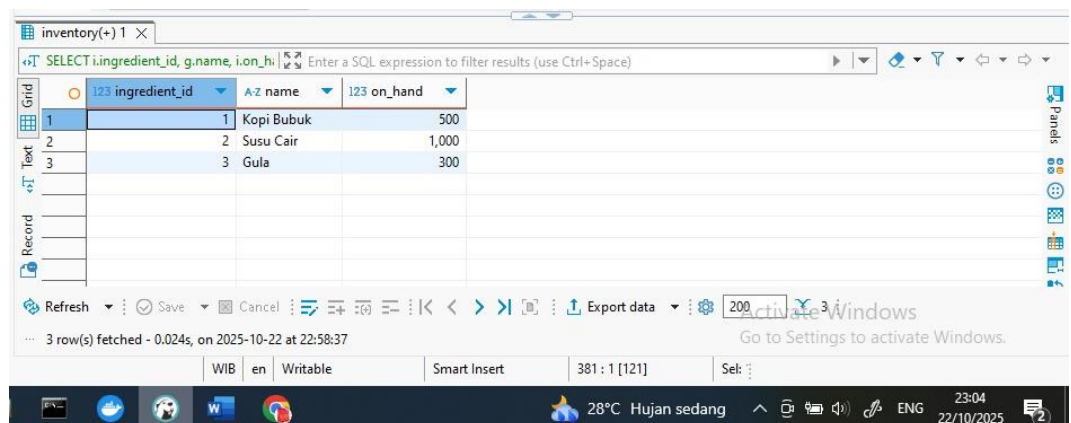
- Trigger AFTER INSERT langsung jalan otomatis:

- Kurangi stok di inventory
- Tambahkan catatan ke inventory_log

Cek hasilnya

Cek stok bahan:

```
SELECT i.ingredient_id, g.name, i.on_hand  
FROM inventory i  
JOIN ingredients g ON g.ingredient_id = i.ingredient_id;
```



ingredient_id	A-Z name	on_hand
1	Kopi Bubuk	500
2	Susu Cair	1,000
3	Gula	300

Artinya stok otomatis berkurang sesuai resep Latte + Espresso.

```
SELECT * FROM inventory_log ORDER BY logged_at DESC;
```

inventory_log 1 X

SQL: `SELECT * FROM inventory_log ORDER BY`

	log_id	ingredient_id	direction	qty	reason	ref_id	logged_at
1	1	1	-1	10	SALE	4	2025-10-22 22:50:16.130 +0700
2	2	2	-1	200	SALE	4	2025-10-22 22:50:16.130 +0700
3	3	3	-1	10	SALE	4	2025-10-22 22:50:16.130 +0700
4	4	1	-1	10	SALE	5	2025-10-22 22:50:16.130 +0700
5	5	3	-1	5	SALE	5	2025-10-22 22:50:16.130 +0700

5 row(s) fetched - 0.240s (0.001s fetch), on 2025-10-22 at 23:06:01

WIB en Writable Smart Insert 386:1 [54] Set

4. payments_ai_set_paid

Trigger ini aktif setiap kali ada data pembayaran masuk di tabel payments.

Jika total pembayaran pelanggan sudah sama atau lebih besar dari total tagihan, maka sistem otomatis mengubah status pesanan di tabel orders menjadi “PAID”.

Selain itu, trigger ini juga mencatat log aktivitas pembayaran ke tabel audit_logs, agar riwayat transaksi tersimpan dengan baik.

DBEaver 25.1.1 - <university_db> tugas.sql

File Edit Navigate Search SQL Editor Database Window Help

Auto university_db public@university_db

Database Navigator Projects

Filter connections by name

- localhost:3306
- railway:ballast.proxy.rlyw.net:58821
- university_db:localhost:5432
 - Databases
 - university_db
 - Schemas
 - coffee_shop_demo
 - Tables
 - audit_logs 16K
 - employees 32K
 - ingredients 48K
 - inventory 40K
 - inventory_log 48K
 - menu_items 32K
 - order_items 40K
 - orders 64K
 - payments 16K
 - recipes 24K
 - shifts 16K
 - suppliers 16K
 - Foreign Tables
 - Views
 - Materialized Views
 - Indexes
 - Functions
 - trg_inventory_apply_recipe()
 - trg_order_items_bd_calc()
 - trg_order_items_biu_calc()
 - trg_orders_bi_build_number()
 - Sequences
 - Data types

Script-2

```
--buat trigger Payments + set order.status = PAID
CREATE OR REPLACE FUNCTION trg_payments_ai_set_paid()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    order_total NUMERIC(12,2);
    paid_sum    NUMERIC(12,2);
BEGIN
    SELECT total_amount INTO order_total FROM orders WHERE order_id = NEW.order_id FOR UPDATE;
    SELECT COALESCE(SUM(amount),0) INTO paid_sum FROM payments WHERE order_id = NEW.order_id;

    IF paid_sum >= order_total THEN
        UPDATE orders SET status = 'PAID' WHERE order_id = NEW.order_id;
    END IF;

    INSERT INTO audit_logs(table_name, action, row_id, details)
    VALUES ('payments','INSERT', NEW.payment_id,
    jsonb_build_object('order_id', NEW.order_id, 'amount', NEW.amount, 'paid_sum', paid_sum));

    RETURN NEW;
END $$;
```

Results 1 X

SQL Error [42P01]: ERROR: relation "payments" does not exist

Go to Error Show log Details >>

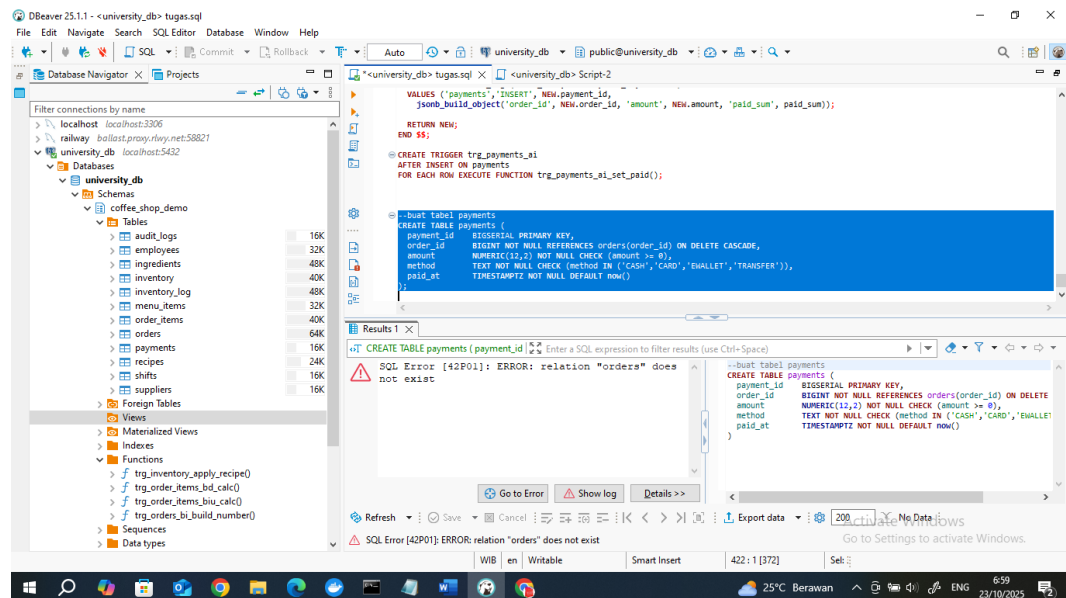
Refresh Save Cancel SQL Editor Database Window Help

WIB en

Disini ada erre: SQL Error [42P01]: ERROR: relation "payments" does not exist

Ini dikarenakan PostgreSQL **tidak menemukan tabel bernama payments** di dalam schema aktif (coffee_shop_demo).

Solusinya membuat tabel payments



Disini ada error lagi ketika membuat tabel payments: SQL Error [42P01]: ERROR: relation "orders" does not exist

Artinya PostgreSQL **nggak nemu tabel orders** di schema aktif tempat dijalankan query-nya.

Solusinya Set schema default dahulu sebelum create table

```
--set schema
SET search_path TO coffee_shop_demo;
```

Lalu jalankan querynya

```
--buat tabel payments
CREATE TABLE payments (
  payment_id BIGSERIAL PRIMARY KEY,
  order_id   BIGINT NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,
  amount     NUMERIC(12,2) NOT NULL CHECK (amount >= 0),
  method     TEXT NOT NULL CHECK (method IN ('CASH', 'CARD', 'EWALLET', 'TRANSFER')),
  paid_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Setelah itu buat triggernya

```
--buat trigger Payments → set order.status = PAID
CREATE OR REPLACE FUNCTION trg_payments_ai_set_paid()
```



```

RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    order_total NUMERIC(12,2);
    paid_sum    NUMERIC(12,2);
BEGIN
    SELECT total_amount INTO order_total FROM orders WHERE order_id = NEW.order_id FOR
    UPDATE;
    SELECT COALESCE(SUM(amount),0) INTO paid_sum FROM payments WHERE order_id =
    NEW.order_id;

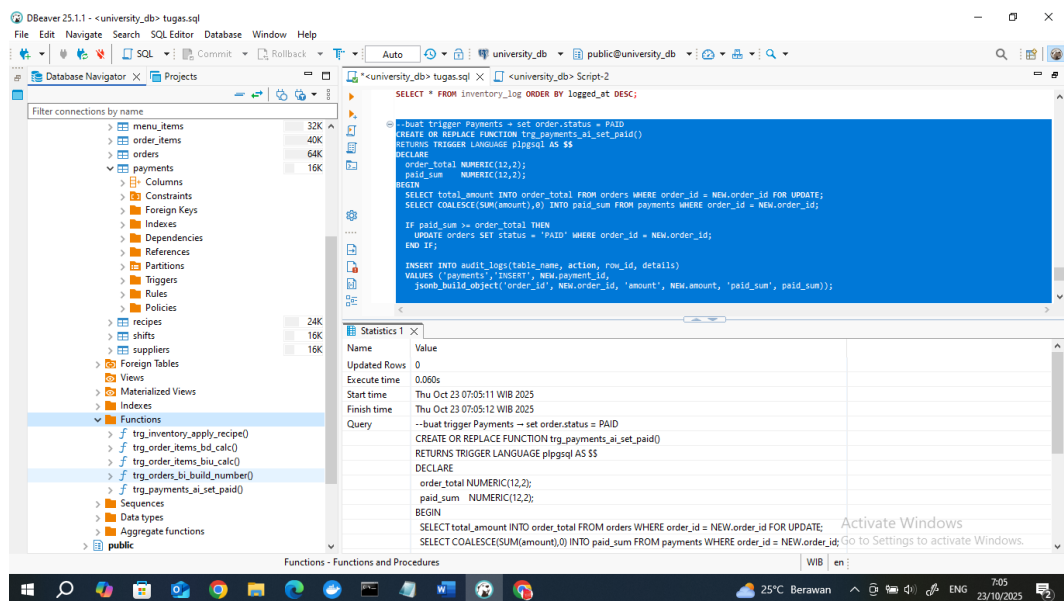
    IF paid_sum >= order_total THEN
        UPDATE orders SET status = 'PAID' WHERE order_id = NEW.order_id;
    END IF;

    INSERT INTO audit_logs(table_name, action, row_id, details)
    VALUES ('payments', 'INSERT', NEW.payment_id,
        jsonb_build_object('order_id', NEW.order_id, 'amount', NEW.amount, 'paid_sum',
        paid_sum));

    RETURN NEW;
END $$;

CREATE TRIGGER trg_payments_ai
AFTER INSERT ON payments
FOR EACH ROW EXECUTE FUNCTION trg_payments_ai_set_paid();

```



Cara penggunaannya:

Pertama masukkan pembayaran:

```

-- masukkan pembayaran
INSERT INTO payments (order_id, amount, method)
VALUES (4, 65000, 'CASH');

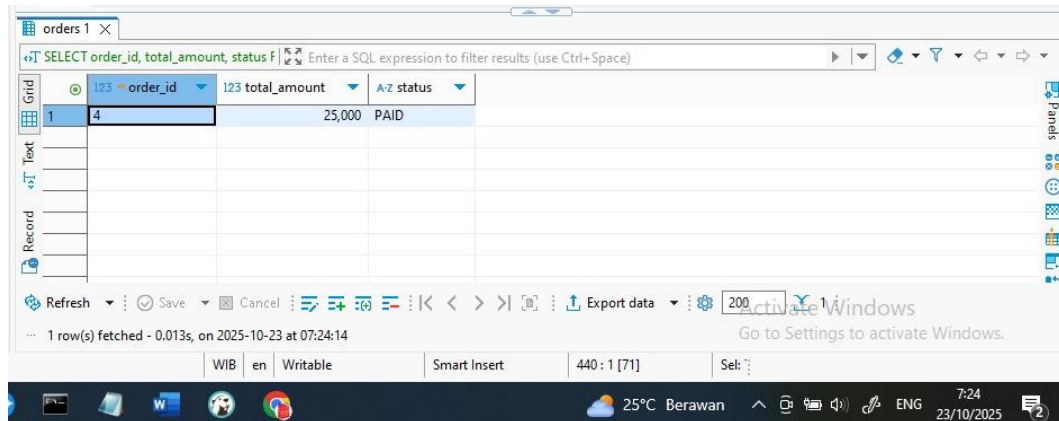
```

Cek hasil:

```

SELECT order_id, total_amount, status FROM orders WHERE order_id = 4;

```



Setelah trigger `trg_payments_ai_set_paid()` dijalankan, sistem akan otomatis memperbarui status order menjadi **PAID** apabila jumlah pembayaran (amount) telah mencapai atau melebihi total harga pesanan (`total_amount`).

E. Views

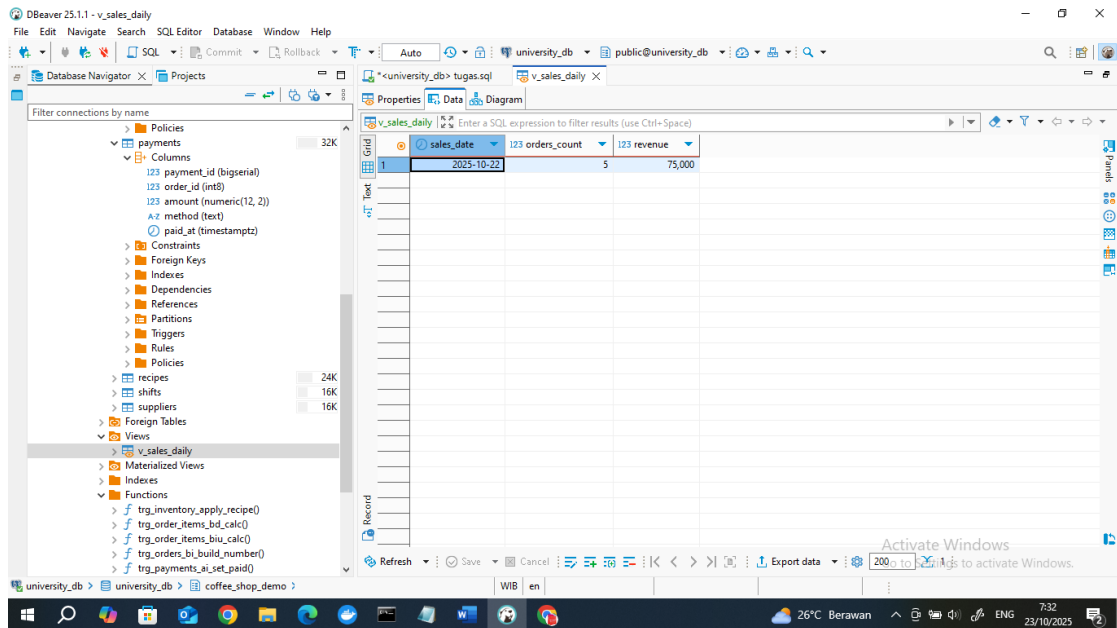
1. `v_sales_daily`

View ini menampilkan ringkasan total penjualan setiap hari.

Isinya mencakup tanggal transaksi, jumlah pesanan yang terjadi, dan total pendapatan pada hari tersebut.

View ini membantu manajer untuk melihat performa penjualan harian tanpa harus menjalankan query kompleks setiap kali.

```
CREATE OR REPLACE VIEW v_sales_daily AS
SELECT
    date_trunc('day', o.order_at)::date AS sales_date,
    COUNT(DISTINCT o.order_id) AS orders_count,
    SUM(o.total_amount) AS revenue
FROM orders o
WHERE o.status <> 'CANCELLED'
GROUP BY 1
ORDER BY 1 DESC;
```



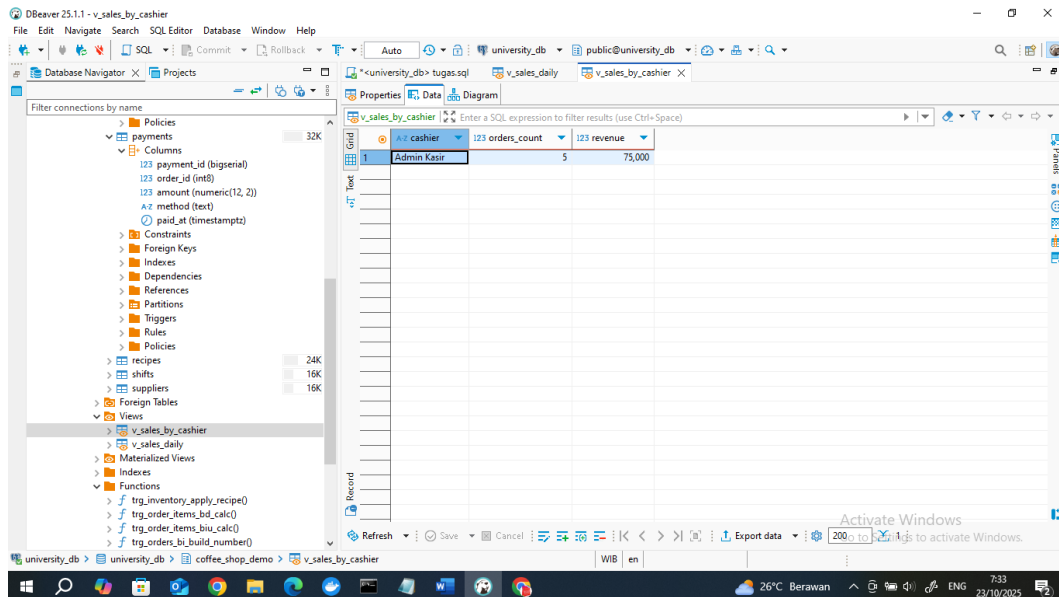
2. v_sales_by_cashier

View ini digunakan untuk melihat performa masing-masing kasir.

Menampilkan jumlah pesanan yang dilayani dan total pendapatan yang dihasilkan oleh setiap kasir.

Dengan view ini, pemilik coffee shop bisa dengan mudah mengevaluasi produktivitas karyawan bagian kasir.

```
CREATE OR REPLACE VIEW v_sales_by_cashier AS
SELECT
    e.full_name AS cashier,
    COUNT(DISTINCT o.order_id) AS orders_count,
    SUM(o.total_amount) AS revenue
FROM orders o
JOIN employees e ON e.employee_id = o.employee_id
WHERE o.status <> 'CANCELLED'
GROUP BY e.full_name
ORDER BY revenue DESC;
```

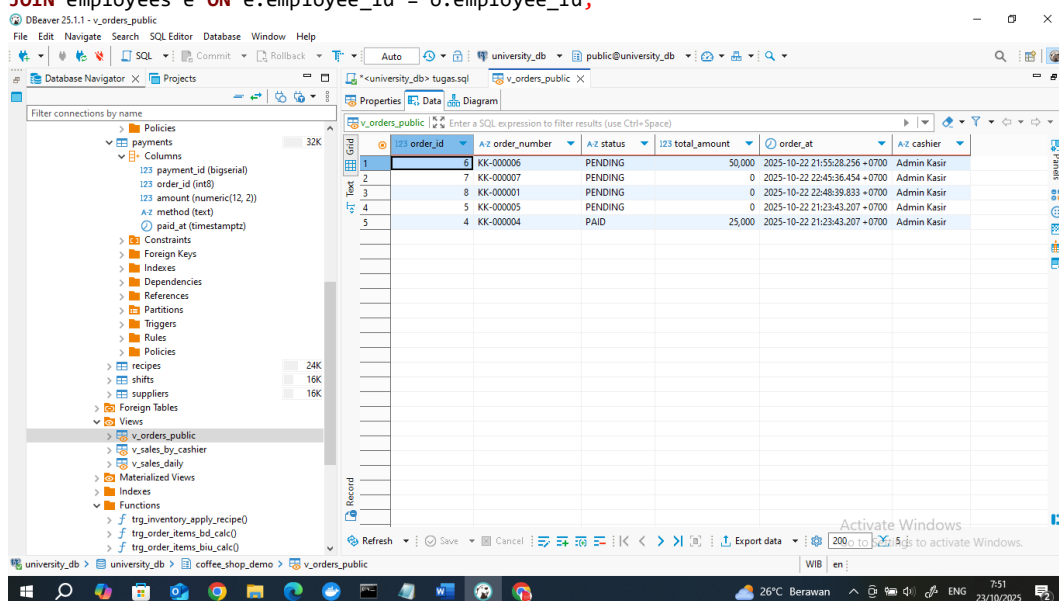


3. v_orders_public

View ini berisi data pesanan dalam bentuk ringkas — seperti nomor pesanan, status, total pembayaran, dan nama kasir.

Fungsinya adalah membatasi akses data agar bagian operasional hanya dapat melihat informasi umum tanpa detail sensitif (misalnya data pelanggan atau rincian bahan baku).

```
CREATE OR REPLACE VIEW v_orders_public AS
SELECT
    o.order_id, o.order_number, o.status, o.total_amount, o.order_at,
    e.full_name AS cashier
FROM orders o
JOIN employees e ON e.employee_id = o.employee_id;
```



F. Materialized View

mv_top_menu_7d

Materialized View ini menampilkan menu yang paling banyak terjual selama 7 hari terakhir, lengkap dengan jumlah penjualan dan total pendapatannya.

Karena sudah disimpan secara fisik di database, hasil query bisa diakses lebih cepat dibandingkan view biasa.

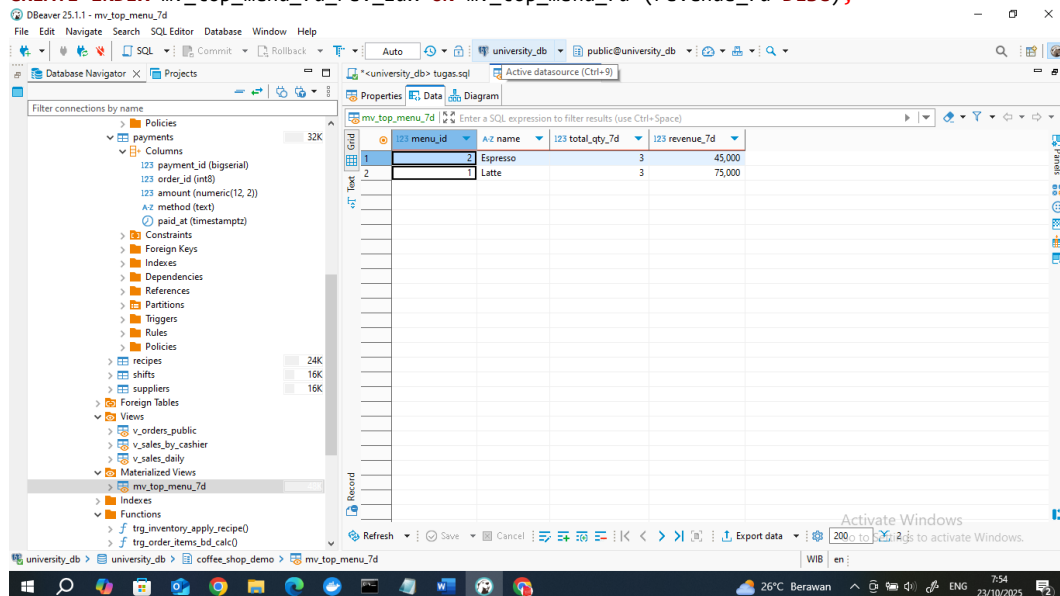
Selain itu, data ini bisa diperbarui secara berkala menggunakan perintah:

```
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_top_menu_7d;
```

Materialized view juga dilengkapi indeks agar performanya lebih optimal saat digunakan di dashboard laporan.

```
CREATE MATERIALIZED VIEW mv_top_menu_7d AS
SELECT
    mi.menu_id,
    mi.name,
    SUM(oi.qty) AS total_qty_7d,
    SUM(oi.subtotal) AS revenue_7d
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
JOIN menu_items mi ON mi.menu_id = oi.menu_id
WHERE o.order_at >= now() - interval '7 days' AND o.status <> 'CANCELLED'
GROUP BY mi.menu_id, mi.name;

CREATE INDEX mv_top_menu_7d_qty_idx ON mv_top_menu_7d (total_qty_7d DESC);
CREATE INDEX mv_top_menu_7d_rev_idx ON mv_top_menu_7d (revenue_7d DESC);
```



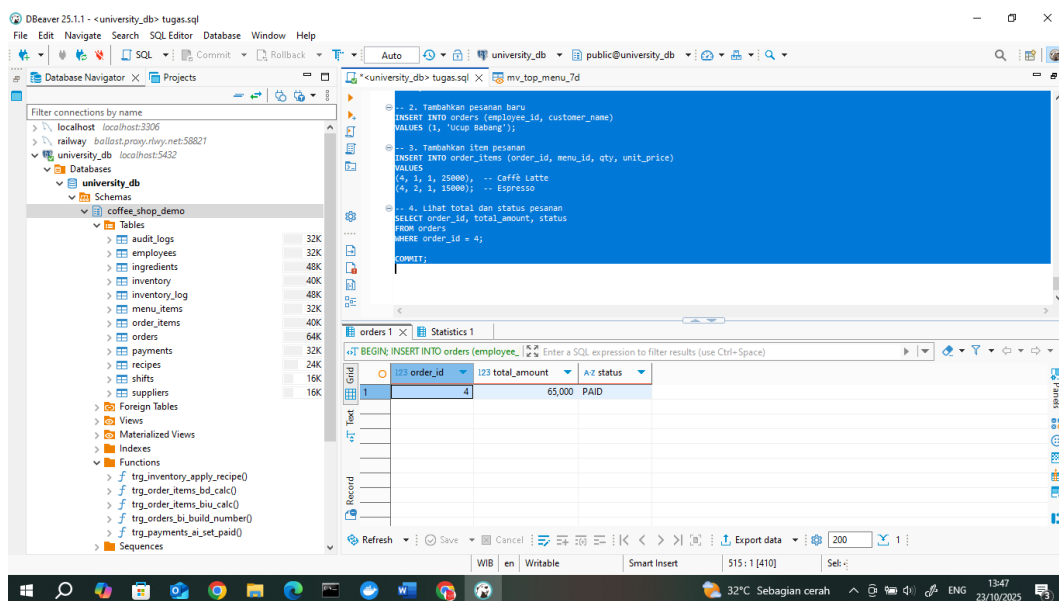
G. Transactions

1. COMMIT – Transaksi Berhasil

Simulasi pertama menunjukkan proses pemesanan 1 Caffè Latte dan 1 Espresso.

Transaksi berjalan lancar, stok bahan otomatis berkurang sesuai resep, pembayaran dilakukan penuh, dan status pesanan berubah menjadi “PAID”.

```
BEGIN;  
  
-- 2. Tambahkan pesanan baru  
INSERT INTO orders (employee_id, customer_name)  
VALUES (1, 'Ucup Babang');  
  
-- 3. Tambahkan item pesanan  
INSERT INTO order_items (order_id, menu_id, qty, unit_price)  
VALUES  
(4, 1, 1, 25000), -- Caffè Latte  
(4, 2, 1, 15000); -- Espresso  
  
-- 4. Lihat total dan status pesanan  
SELECT order_id, total_amount, status  
FROM orders  
WHERE order_id = 4;  
  
COMMIT;
```



Skenario:

Customer “Ucup Babang” memesan 1 Caffè Latte dan 1 Espresso.
Pembayaran dilakukan penuh (Rp65.000).

Hasil Implementasi:

- Transaksi berhasil disimpan.
- Stok bahan otomatis berkurang sesuai resep.
- Total order dihitung otomatis (Caffè Latte + Espresso = Rp65.000).

- Status pesanan berubah dari *PENDING* → *PAID*.
- Perubahan tersimpan permanen setelah COMMIT.

Trigger menghitung otomatis:

- `order_items_biu_calc()` → isi subtotal.
- `orders.total_amount` diupdate jadi **65.000**.
- `trg_inventory_apply_recipe()` → stok bahan berkurang otomatis.
- `trg_payments_ai_set_paid()` → status jadi **PAID** kalau pembayaran sudah cukup.

2. ROLLBACK – Transaksi Gagal

Simulasi ini menguji mekanisme **ROLLBACK** otomatis ketika stok bahan tidak cukup.

Kita akan coba memesan 300 Caffè Latte, padahal stok susu (ingredient Susu Cair) cuma 1000 ml.

Trigger `trg_inventory_apply_recipe()` bakal mendeteksi stok kurang dan mengeluarkan error *Insufficient stock*, lalu PostgreSQL otomatis melakukan ROLLBACK, artinya semua perubahan dibatalkan agar data tetap konsisten.

```
--ROLLBACK -Transaksi gagal
SET search_path TO coffee_shop_demo;

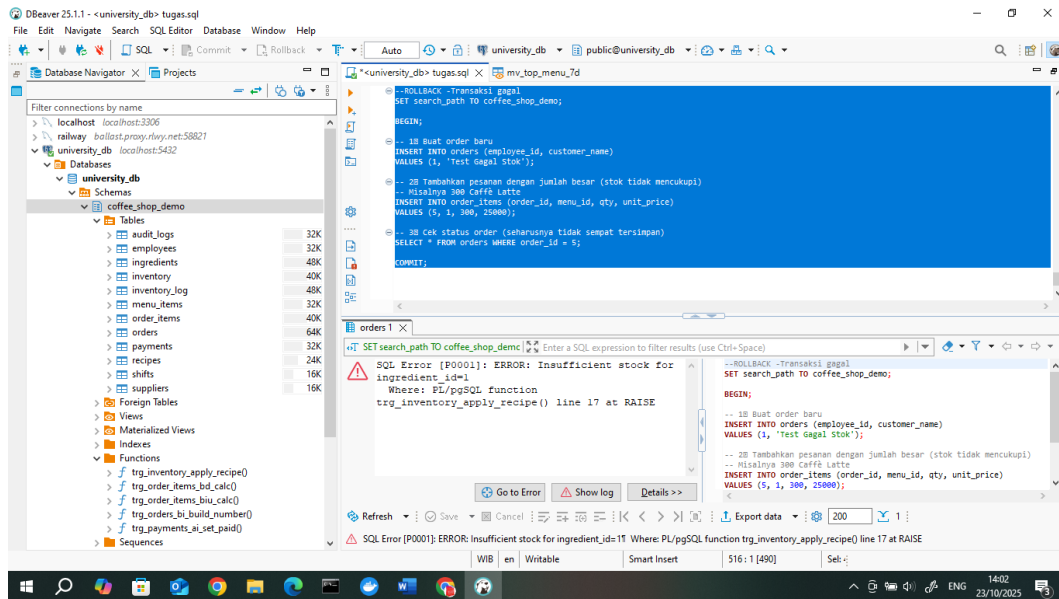
BEGIN;

-- 1]Buat order baru
INSERT INTO orders (employee_id, customer_name)
VALUES (1, 'Test Gagal Stok');

-- 2]Tambahkan pesanan dengan jumlah besar (stok tidak mencukupi)
-- Misalnya 300 Caffè Latte
INSERT INTO order_items (order_id, menu_id, qty, unit_price)
VALUES (5, 1, 300, 25000);

-- 3]Cek status order (seharusnya tidak sempat tersimpan)
SELECT * FROM orders WHERE order_id = 5;

COMMIT;
```



Hasilnya error:

SQL Error [P0001]: ERROR: Insufficient stock for ingredient_id=1

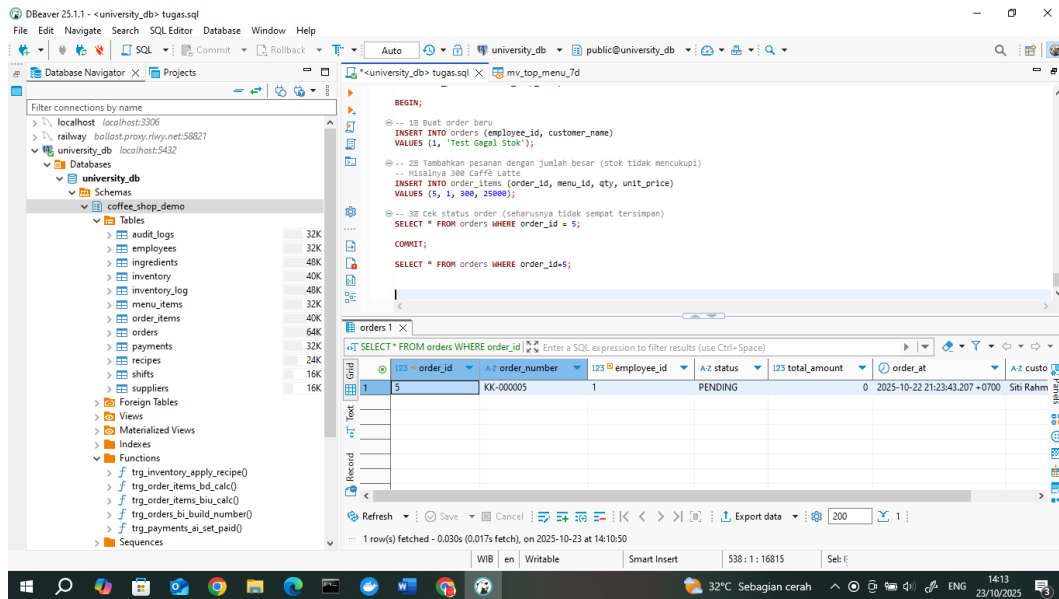
Where: PL/pgSQL function trg_inventory_apply_recipe() line 17 at RAISE

Artinya:

- Trigger trg_inventory_apply_recipe() mendeteksi bahwa stok bahan (ingredient_id=1 = susu cair) tidak mencukupi untuk memenuhi pesanan “300 Caffè Latte”.
- Begitu trigger ini menjalankan RAISE EXCEPTION 'Insufficient stock...', PostgreSQL langsung membatalkan seluruh transaksi aktif (ROLLBACK otomatis).
- Karena masih di dalam blok BEGIN ... COMMIT, sistem nggak nyimpen order baru dan nggak ada perubahan stok.

Hasilnya setelah di check

SELECT * FROM orders WHERE order_id=5;



Hasil akhir menunjukkan bahwa pesanan dengan order_id=5 memiliki status “PENDING” dan total_amount bernilai 0, menandakan transaksi gagal tersimpan dengan sempurna sesuai mekanisme atomicity pada prinsip ACID.

H. Analisis & Perbandingan Approach

Tujuan Umum

Tujuan dari implementasi ini adalah memastikan **konsistensi data transaksi** pada sistem *Coffee Shop Demo*, khususnya saat terjadi proses pemesanan, pembayaran, dan pengurangan stok bahan baku.

Sistem harus mematuhi prinsip ACID (Atomicity, Consistency, Isolation, Durability), di mana setiap transaksi:

- Hanya akan tersimpan jika seluruh proses sukses (COMMIT),
- Dan akan dibatalkan total jika salah satu bagian gagal (ROLLBACK).

1. Analisis Pendekatan yang Digunakan (Current Approach)

- Pendekatan: Trigger-Based Transaction Control

Pada tugas ini, digunakan kombinasi:

- Trigger PL/pgSQL untuk otomatisasi logika bisnis (update stok, audit log, status order, dsb),

- Transaction Block (BEGIN ... COMMIT / ROLLBACK) untuk mengelola keberhasilan atau kegagalan transaksi.

Implementasi Teknis:

Fitur	Implementasi	Fungsi
Trigger trg_inventory_apply_recipe()	Mengecek stok bahan saat order item ditambahkan	Menjamin stok tidak minus; jika stok kurang → <i>raise exception</i>
Trigger trg_payments_ai_set_paid()	Mengubah status order jadi <i>PAID</i> jika total pembayaran terpenuhi	Menjaga sinkronisasi antara tabel payments dan orders
Transaction Control	BEGIN; ... COMMIT; dan otomatis ROLLBACK saat error	Menjaga atomicity (semua atau tidak sama sekali)
Audit Logs	Mencatat aktivitas INSERT, UPDATE, dan DELETE	Mendukung traceability dan debugging

Kelebihan Pendekatan Ini

1. Otomatis & Konsisten
Semua logika validasi dan update berjalan otomatis lewat trigger tanpa harus ditulis manual di tiap transaksi.
2. Mendukung Atomicity dan Consistency (AC dari ACID)
Jika salah satu operasi gagal (contoh: stok habis), trigger me-*raise exception* → sistem langsung rollback seluruh transaksi.
3. Minim Human Error
Developer tidak perlu memikirkan update stok atau status order secara manual, karena sudah otomatis dikelola trigger.
4. High Reliability untuk Sistem Nyata
Cocok untuk sistem POS (*Point of Sales*) yang butuh integritas tinggi antara transaksi dan stok barang.

Kekurangan Pendekatan Ini

1. Kompleksitas Debugging Tinggi
Karena error sering muncul dari dalam trigger, sulit melacak penyebabnya tanpa menelusuri fungsi PL/pgSQL.
2. Performansi Bisa Turun pada Volume Tinggi
Banyak trigger aktif dapat memperlambat *insert/update* saat transaksi masif (misal ratusan pesanan per menit).
3. Kurang Fleksibel untuk Custom Flow
Semua logika otomatis — susah dilakukan override tanpa ubah kode trigger.

Analisis Hasil Implementasi

Skenario	Deskripsi	Hasil	Status
COMMIT – Transaksi Berhasil	Pemesanan 1 Caffè Latte dan 1 Espresso, pembayaran penuh	Status berubah ke <i>PAID</i> , stok berkurang otomatis	☒ Sukses
ROLLBACK – Transaksi Gagal	Pemesanan 300 Caffè Latte, stok tidak mencukupi	Trigger menolak transaksi → rollback otomatis	☒ Sukses
Konsistensi Data	Cek tabel <code>orders</code> , <code>order_items</code> , dan <code>inventory</code>	Tidak ada data parsial atau stok negatif	☒ Terjaga

Kesimpulan

Pendekatan yang digunakan telah berhasil menunjukkan implementasi nyata dari konsep:

- Atomicity: Semua operasi sukses → commit, gagal → rollback total.
- Consistency: Integritas data antar tabel tetap terjaga.
- Isolation: Transaksi berjalan independen dari transaksi lain.
- Durability: Data tersimpan permanen setelah commit.

Sistem *Coffee Shop Demo* dengan trigger dan transaksi ini membuktikan bahwa PostgreSQL mampu menjadi fondasi kuat untuk sistem real-time transactional database yang reliabel dan efisien.

Dengan memanfaatkan mekanisme trigger dan transaction control di PostgreSQL, sistem mampu menjaga integritas data secara otomatis tanpa intervensi manual. Pendekatan ini membuktikan prinsip ACID bekerja dengan optimal pada kasus simulasi transaksi Coffee Shop.